



Aplicaciones e Industria de IoT

Semana 4 — Smart Manufacturing e IIoT

KOICA

IGU HANDONG GLOBAL
UNIVERSITY



UNA

ÍNDICE

- 1. Introducción a IIoT e Industria 4.0**
- 2. Convergencia IT/OT y Sistemas Industriales**
- 3. Monitoreo de Activos Industriales con IoT**
- 4. KPIs Industriales: OEE, MTBF, MTTR**
- 5. Mantenimiento: Del Correctivo al Predictivo**
- 6. Casos de Éxito en la Industria Real**
- 7. Laboratorio PBL**
- 8. Resumen, Bibliografía y Próximos Pasos**



Repaso de la Semana 3 y Objetivos de la Semana 4



Lo que aprendimos en la Semana 3

- Tipos de datos IoT: series temporales, eventos, estados, multimedia.
- Pipeline de datos: ingesta → almacenamiento → procesamiento → visualización.
- Analítica: descriptiva, diagnóstica, predictiva, prescriptiva.
- Machine Learning aplicado: regresión, Random Forest, K-Means, LSTM.
- Ejercicio: ESP32 + DHT22 + MQTT generando datos en Wokwi.
- LAB PBL: Pipeline completo de cadena de frío con Python.



Objetivos de la Semana 4

- Comprender IIoT y su diferencia con IoT de consumo.
- Analizar la convergencia IT/OT y los protocolos industriales.
- Diseñar estrategias de monitoreo de activos críticos.
- Calcular KPIs industriales: OEE, MTBF, MTTR, disponibilidad.
- Diferenciar mantenimiento correctivo, preventivo y predictivo.
- Implementar monitoreo de vibración y temperatura con ESP32.



Conexión: La Semana 3 diseñamos el pipeline de datos. Hoy lo aplicamos a manufactura y operaciones industriales

4

Aplicación
HOY: Smart Manufacturing

3

Plataforma / Datos
(Semana 3: Pipeline y ML)

2

Red
(Semana 2: Protocolos)

1

Dispositivo
(Semana 1: ESP32)

1. INTRODUCCIÓN A IIoT E INDUSTRIA 4.0

¿Qué es el IIoT? – Internet de las Cosas Industrial

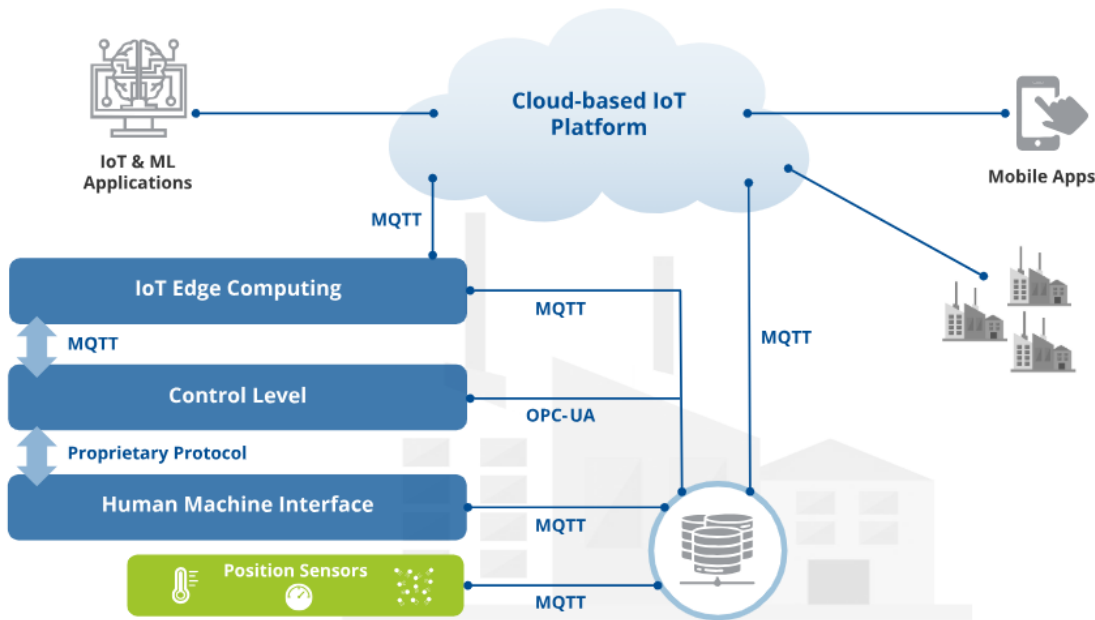
El IIoT (Industrial Internet of Things) es la aplicación de sensores, conectividad y analítica de datos en entornos industriales para mejorar la eficiencia operativa, reducir costos y crear nuevos modelos de negocio. A diferencia del IoT de consumo (domótica, wearables), el IIoT opera en condiciones extremas con requisitos estrictos de confiabilidad, seguridad y disponibilidad.



IoT de Consumo vs. IIoT – Diferencias Fundamentales

Criterio	IoT de Consumo	IIoT (Industrial)
Entorno	Hogar, oficina, ciudad	Fábrica, planta, mina, campo
Tolerancia a fallos	Baja (inconveniente)	Crítica (pérdidas, riesgos)
Volumen de datos	Moderado	Masivo (miles de sensores)
Latencia requerida	Segundos aceptables	Milisegundos (control)
Seguridad	Privacidad personal	Seguridad física + cibernética
Ciclo de vida	2-5 años	10-30 años
Ejemplo	Termostato Nest, Alexa	Sensor de vibración en turbina

¿Qué es el IIoT? – Internet de las Cosas Industrial



Arquitectura IIoT

- **Arquitectura IIoT en capas:** los sensores de campo capturan datos y los envían vía MQTT al HMI (visualización local) y al Edge (preprocesamiento). La capa de control (PLCs) usa OPC-UA para interoperar con la nube. La plataforma cloud centraliza todo y alimenta aplicaciones de ML, dashboards móviles y plantas remotas. MQTT es el protocolo dominante en todo el flujo; OPC-UA conecta específicamente el mundo OT con IT.

Las 4 Revoluciones Industriales – Del Vapor al IIoT



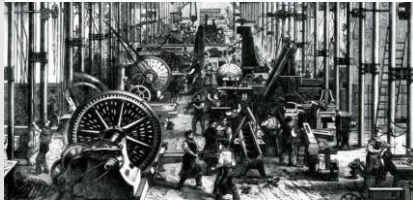
Evolución Histórica de la Manufactura

1.0

~1784

Mecanización

Máquina de vapor, telares mecánicos. Producción manual → mecanizada. Primera fábrica textil.

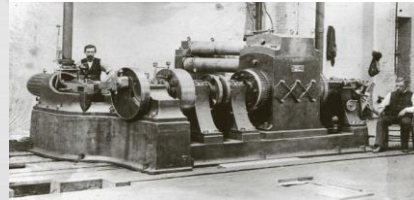


2.0

~1870

Electrificación

Electricidad, línea de ensamblaje (Ford). Producción en masa, división del trabajo.



3.0

~1969

Automatización

PLC, robots, computadoras industriales. Automatización programable de procesos.



4.0

~2011

Digitalización

IIoT, ciber-físicos, IA, gemelos digitales. Fábricas inteligentes, decisiones autónomas basadas en datos.



💡 El término "Industrie 4.0" fue acuñado en la Feria de Hannover en 2011 por el gobierno alemán (Kagermann, Wahlster & Helbig, 2013). Describe la transformación de la manufactura mediante sistemas ciber-físicos (CPS), IoT y computación en la nube.

2. CONVERGENCIA IT/OT Y SISTEMAS INDUSTRIALES

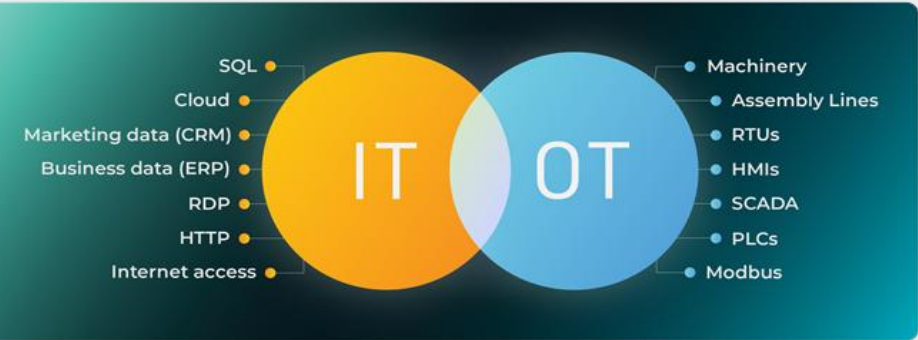
Convergencia IT/OT – Dos Mundos que se Encuentran

IT – Tecnologías de la Información

Sistemas empresariales: ERP, CRM, bases de datos, redes corporativas, cloud computing.

Características:

- Ciclos de actualización cortos (meses).
- Prioridad: confidencialidad de datos.
- Protocolos estándar: TCP/IP, HTTP, SQL.
- Personal: ingenieros de sistemas, DBAs.



OT – Tecnologías Operativas

Sistemas de planta: SCADA, PLC, DCS, HMI, robots industriales, controladores.

Características:

- Ciclos de vida largos (10-30 años).
- Prioridad: disponibilidad y seguridad física.
- Protocolos propietarios: Modbus, Profinet, OPC.
- Personal: Ing. eléctricos, electrónicos, automatización.

 **IIoT es el puente: conecta los datos de planta (OT) con los sistemas de gestión (IT) para tomar decisiones basadas en datos reales**

La convergencia IT/OT permite que un gerente de planta vea en tiempo real la eficiencia de cada máquina desde su ERP, o que un algoritmo de ML detecte fallas inminentes usando datos de sensores de vibración combinados con órdenes de producción del MES.

SCADA, PLC, DCS y MES – Los Pilares de la Automatización



Sistemas de Automatización Industrial – Glosario Esencial

PLC

Programmable Logic Controller

Controlador industrial programable que ejecuta lógica de control en tiempo real. Lee sensores, ejecuta algoritmos y activa actuadores. Ejemplo: controlar la velocidad de una cinta transportadora según el peso detectado.

SCADA

Supervisory Control and Data Acquisition

Sistema de supervisión centralizado que recolecta datos de múltiples PLCs/RTUs, los visualiza en pantallas (HMI) y permite control remoto. Ejemplo: monitorear todas las subestaciones eléctricas de una red desde un centro de control.

DCS

Distributed Control System

Sistema de control distribuido para procesos continuos (petroquímica, generación eléctrica). Similar a SCADA pero con control más integrado. Los controladores están distribuidos físicamente en la planta.

MES

Manufacturing Execution System

Sistema que conecta el piso de planta con la gestión empresarial (ERP). Rastrea órdenes de producción, materiales, tiempos, calidad y eficiencia en tiempo real. Puente entre IT y OT.

Protocolos Industriales – OPC-UA, Modbus y Profinet



¿Cómo se Comunican los Equipos en una Planta Industrial?

OPC-UA

Open Platform Communications – Unified Architecture. Estándar abierto, multiplataforma, seguro (TLS). Modelo de datos jerárquico. Es el protocolo estrella de Industria 4.0: permite que un PLC Siemens hable con un servidor AWS IoT.

Modbus

Protocolo serial creado en 1979 por Modicon. Simple, confiable, ampliamente soportado. Variantes: Modbus RTU (serial RS-485) y Modbus TCP (Ethernet). Limitación: sin seguridad nativa ni modelo de datos jerárquico.

Profinet

Protocolo Ethernet industrial de Siemens. Comunicación determinista en tiempo real. Tres niveles: TCP/IP (no crítico), RT (tiempo real suave) e IRT (tiempo real duro, ciclos de 31.25 μ s). Dominante en fábricas con equipos Siemens.

MQTT + Sparkplug B

MQTT con estructura de datos estandarizada para IIoT. Sparkplug define el formato de payload, autodescubrimiento de dispositivos y gestión de estado. Permite integrar equipos OT con plataformas cloud usando infraestructura MQTT existente.

3. MONITOREO DE ACTIVOS INDUSTRIALES CON IoT

¿Qué es un Activo Industrial? – Definición y Clasificación

Un activo industrial es cualquier equipo, máquina, infraestructura o componente que tiene valor económico y es esencial para la operación de una planta o proceso productivo. Monitorear la condición de estos activos en tiempo real con sensores IoT permite anticipar fallas, optimizar el uso y extender su vida útil.



Clasificación de Activos por Criticidad

Activos Críticos

Si fallan, la producción se detiene completamente. Ejemplo: compresor principal, horno de fundición, transformador de potencia. Requieren monitoreo continuo 24/7.

Activos Importantes

Su falla degrada la producción pero no la detiene. Ejemplo: bombas secundarias, ventiladores de enfriamiento. Monitoreo periódico o por condición.

Activos de Soporte

Equipos auxiliares cuya falla tiene impacto menor. Ejemplo: iluminación de planta, sistemas HVAC de oficinas. Mantenimiento programado estándar.

Regla 80/20: típicamente el 20% de los activos genera el 80% del riesgo operativo. El monitoreo IoT debe priorizar ese 20% crítico.

Sensores para Monitoreo de Activos Industriales



Los 6 Tipos de Sensores más Usados en IIoT

Vibración

Acelerómetro (MEMS/piezoeléctrico)

Detectar desbalance, desalineamiento, rodamientos dañados en motores, bombas, ventiladores.

Temperatura

Termopar, RTD, infrarrojo

Monitorear sobrecalentamiento de motores, transformadores, cojinetes, tableros eléctricos.

Corriente eléctrica

Transformador de corriente (CT)

Detectar sobrecarga, fases desbalanceadas, arranques anormales en motores y equipos eléctricos.

Presión

Piezorresistivo, capacitivo

Monitorear presión de sistemas hidráulicos, neumáticos, calderas, compresores.

Ultrasonido

Micrófono ultrasónico

Detectar fugas de aire comprimido, descargas parciales en equipos de alta tensión, fricción.

Análisis de aceite

Sensor de partículas, viscosidad

Evaluar desgaste interno de cajas de engranajes, turbinas, motores diésel sin desarmar.

Ejercicio – Monitoreo de Motor Industrial en Wokwi

EJERCICIO EN CLASE

 **Objetivo:** Simular el monitoreo de un motor industrial con sensor de temperatura y alerta por sobrecalentamiento

Instrucciones Paso a Paso

1. Abrir wokwi.com → ESP32 → nuevo proyecto.
2. Agregar DHT22: VCC→3V3, GND→GND, DATA→GPIO4.
3. Agregar 3 LEDs con resistencias de 220Ω:
Verde (GPIO25), Amarillo (GPIO26), Rojo (GPIO27).
4. En Libraries agregar: DHT sensor library + PubSubClient.
5. Copiar código de la siguiente diapositiva.
6. Ejecutar y verificar en Serial Monitor:
 - LED verde = Normal (<60°C)
 - LED amarillo = Advertencia (60-75°C)
 - LED rojo = Alarma crítica (>75°C)
7. Abrir MQTTX → suscribirse al topic.

¿Qué Van a Aprender?

- Cómo simular estados de un motor (normal, advertencia, alarma) usando umbrales de temperatura.
- Indicadores visuales (LEDs) como feedback inmediato en el borde.
- Publicación de telemetría con estado calculado vía MQTT.
- Lógica de 3 niveles típica de alarmas industriales.

Datos del Broker

Servidor: `broker.hivemq.com`
Puerto: 1883 (sin cifrado)
Topic: `iot-fpuna/sem4/motor/TU-NOMBRE`

Ejercicio · Código Arduino IDE – Monitoreo de Motor



Código Completo – Copiar en sketch.ino de Wokwi

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT22
#define LED_G 25 // Normal
#define LED_Y 26 // Advertencia
#define LED_R 27 // Alarma

const char* ssid = "Wokwi-GUEST";
const char* pass = "";
const char* broker = "broker.hivemq.com";
const char* topic = "iot-fpuna/sem4/motor/planta1";
const char* motor_id = "MOTOR-COMPRESOR-01";

WiFiClient espClient;
PubSubClient mqtt(espClient);
DHT dht(DHTPIN, DHTTYPE);
int seq = 0;

void setup() {
  Serial.begin(115200); dht.begin();
  pinMode(LED_G, OUTPUT);
  pinMode(LED_Y, OUTPUT);
  pinMode(LED_R, OUTPUT);
  WiFi.begin(ssid, pass);
  while(WiFi.status() != WL_CONNECTED){delay(500);}
  Serial.println("WiFi OK");
  mqtt.setServer(broker, 1883);
}

void reconectar(){
  while(!mqtt.connected()){
    if(mqtt.connect(motor_id))
      Serial.println("MQTT OK");
    else delay(3000);
  }
}

void loop() {
  if(!mqtt.connected()) reconectar();
  mqtt.loop();
  float t=dht.readTemperature();
  float h=dht.readHumidity();
  if(isnan(t)){delay(2000);return;}
  seq++;
  // Lógica de 3 niveles
  const char* estado;
  if(t>=75.0){estado="ALARMA";
    digitalWrite(LED_G, LOW);
    digitalWrite(LED_Y, LOW);
    digitalWrite(LED_R, HIGH);
  }else if(t>=60.0){estado="ADVERTENCIA";
    digitalWrite(LED_G, LOW);
    digitalWrite(LED_Y, HIGH);
    digitalWrite(LED_R, LOW);
  }else{estado="NORMAL";
    digitalWrite(LED_G, HIGH);
    digitalWrite(LED_Y, LOW);
    digitalWrite(LED_R, LOW);
  }
  char json[250];
  sprintf(json, 250,
    "{\n  \"motor_id\": \"%s\", \"\n  \"temp_c\": %.1f, \"hum\": %.1f, \"\n  \"estado\": \"%s\", \"seq\": %d\", \n  \"motor_id\", t, h, estado, seq);
  mqtt.publish(topic, json);
  Serial.println(json);
  delay(3000);
}
```

Notas Clave

Lógica de 3 niveles:

Replica la clasificación estándar de alarmas industriales ISA-18.2:

NORMAL: $t < 60^{\circ}\text{C}$

Motor operando dentro de parámetros. LED verde encendido.

ADVERTENCIA: $60-75^{\circ}\text{C}$

Temperatura elevada. Requiere atención. LED amarillo.

ALARMA: $t \geq 75^{\circ}\text{C}$

Riesgo de daño. Acción inmediata requerida. LED rojo.

broker.hivemq.com

Broker MQTT público gratuito. Funciona desde Wokwi sin problemas.

4. KPIs INDUSTRIALES OEE • MTBF • MTTR • DISPONIBILIDAD

KPIs Industriales — ¿Qué Medimos y Por Qué?

Un KPI (Key Performance Indicator) es un indicador clave de rendimiento que permite medir objetivamente si un proceso, equipo o planta está cumpliendo sus objetivos operativos. En manufactura, los KPIs conectan la realidad del piso de planta con las decisiones gerenciales. "Lo que no se mide, no se puede mejorar" — Peter Drucker.



Los 4 KPIs Fundamentales de Manufactura

OEE

Overall Equipment Effectiveness

Clase mundial: $\geq 85\%$

Mide la eficiencia global del equipo combinando disponibilidad $\times$ rendimiento $\times$ calidad. Es el KPI más completo.

MTBF

Mean Time Between Failures

Objetivo: maximizar

Tiempo promedio entre fallas. Cuanto mayor, más confiable es el equipo. Se calcula: tiempo operativo total / número de fallas.

MTTR

Mean Time To Repair

Objetivo: minimizar

Tiempo promedio de reparación. Cuanto menor, más rápido el equipo vuelve a producir. Incluye diagnóstico + reparación + pruebas.

Disponibilidad

Availability

Objetivo: $\geq 95\%$

Porcentaje del tiempo que el equipo está operativo. Fórmula: $\text{MTBF} / (\text{MTBF} + \text{MTTR}) \times 100$. Combina confiabilidad con mantenibilidad.

OEE – El Indicador Estrella de la Manufactura

 $OEE = \text{Disponibilidad} \times \text{Rendimiento} \times \text{Calidad}$

Disponibilidad

Tiempo operativo / Tiempo planificado

Pérdidas típicas:

Paradas no planificadas, cambios de producto, arranques, fallas de equipo.

90%

Rendimiento

Producción real / Producción teórica

Pérdidas típicas:

Microparadas, velocidad reducida, ciclos lentos, atascos menores.

95%

Calidad

Piezas buenas / Total producido

Pérdidas típicas:

Piezas defectuosas, retrabajo, scrap, rechazos de calidad.

99%

Ejemplo de cálculo:

Un turno de 8 horas planificado. El equipo operó 7.2h (0.8h de paradas) → Disponibilidad = $7.2/8 = 90\%$. Produjo 950 de 1000 piezas teóricas → Rendimiento = 95%. De las 950, 941 fueron buenas → Calidad = 99.1%. $OEE = 0.90 \times 0.95 \times 0.991 = 84.7\%$. Resultado: muy cerca de clase mundial (85%), pero las paradas no planificadas son el punto de mejora.

MTBF y MTTR – Confiabilidad y Mantenibilidad



MTBF – Tiempo Medio entre Fallas

Fórmula:

$\text{MTBF} = \text{Tiempo operativo total} / \text{Número de fallas}$

Ejemplo: Un motor operó 2.000 horas y tuvo 4 fallas.

$\text{MTBF} = 2.000 / 4 = 500 \text{ horas}$

Interpretación: en promedio, el motor funciona 500 horas antes de la próxima falla. A mayor MTBF, mayor confiabilidad.

IoT mejora MTBF porque permite detectar condiciones de deterioro antes de que se conviertan en fallas.



MTTR – Tiempo Medio de Reparación

Fórmula:

$\text{MTTR} = \text{Tiempo total de reparaciones} / \text{Número de fallas}$

Ejemplo: Las 4 fallas del motor sumaron 12 horas de reparación.

$\text{MTTR} = 12 / 4 = 3 \text{ horas}$

Incluye: diagnóstico + obtención de repuestos + reparación + pruebas de verificación.

IoT reduce MTTR porque el sistema indica exactamente qué componente falló y cuál es la causa probable.



$\text{Disponibilidad} = \text{MTBF} / (\text{MTBF} + \text{MTTR}) \times 100 = 500 / (500 + 3) \times 100 = 99.4\%$

La disponibilidad combina ambos indicadores. Un equipo puede ser muy confiable (MTBF alto) pero si cuando falla tarda mucho en repararse (MTTR alto), su disponibilidad baja. IoT mejora ambos: detecta fallas temprano (↑MTBF) y facilita diagnóstico rápido (↓MTTR).

5. MANTENIMIENTO DEL CORRECTIVO AL PREDICTIVO

Evolución del Mantenimiento Industrial



Los 4 Niveles de Madurez del Mantenimiento

1

Correctivo

"Reparar cuando falla"

Se interviene solo cuando el equipo deja de funcionar. Máximo tiempo de parada, costos impredecibles, riesgos de seguridad. Es el nivel más básico y costoso a largo plazo.

Costo: 100%
(referencia)

2

Preventivo

"Reparar por calendario"

Intervenciones programadas a intervalos fijos (cada 1.000h, cada 6 meses). Reduce paradas imprevistas pero puede generar sobre-mantenimiento: cambiar piezas que aún estaban buenas.

Costo: ~60-80%

3

Predictivo

"Reparar cuando los datos lo indican"

Monitoreo continuo de condición (vibración, temperatura, corriente). ML predice cuándo fallará. Se interviene justo antes de la falla. Máxima vida útil de componentes.

Costo: ~30-50%

4

Prescriptivo

"El sistema decide qué hacer"

El sistema no solo predice la falla sino que recomienda la acción: qué reparar, cuándo, con qué repuestos, quién debe hacerlo. Automatización total de la gestión de mantenimiento.

Costo: ~20-40%

Mantenimiento Predictivo con IoT – El Pipeline Completo

De los Sensores a la Orden de Trabajo Automática



Beneficios Comprobados del Mantenimiento Predictivo

30-50%

Reducción de paradas imprevistas

20-40%

Extensión de vida útil de equipos

10-25%

Ahorro en costos de mantenimiento

35-45%

Reducción de inventario de repuestos

Gemelo Digital – La Réplica Virtual de tu Activo

Un gemelo digital (Digital Twin) es una réplica virtual de un activo físico, proceso o sistema que se actualiza en tiempo real con datos de sensores IoT. Permite simular escenarios, predecir comportamientos y optimizar operaciones sin intervenir el equipo real. El concepto fue formalizado por la NASA en 2010 y hoy es adoptado por toda la industria.



Cómo Funciona el Gemelo Digital

1. Modelar: crear un modelo 3D o matemático del activo con sus parámetros de diseño.
2. Conectar: vincular sensores IoT reales al modelo. Cada lectura actualiza el gemelo.
3. Simular: ejecutar escenarios "qué pasaría si": ¿qué ocurre si aumento la carga 20%? ¿si no hago mantenimiento en 2 semanas?
4. Optimizar: ajustar parámetros del activo real basándose en las simulaciones del gemelo digital.
5. Predecir: el gemelo anticipa fallas y recomienda acciones basado en el modelo + datos históricos + ML.



Aplicaciones Industriales

- Manufactura: gemelos de líneas de producción para optimizar layout, tiempos de ciclo y flujo de materiales (Siemens, GE).
- Energía: gemelos de turbinas eólicas para predecir generación y planificar mantenimiento (Vestas, GE Renewable).
- Petróleo: gemelos de plataformas offshore para simular condiciones extremas antes de operar (Shell, BP).
- Logística: gemelos de flotas de vehículos para optimizar rutas y consumo (DHL, UPS).
- Edificios: gemelos de edificios inteligentes para gestionar energía y confort (Microsoft, Willow).

6. CASOS DE ÉXITO EN LA INDUSTRIA REAL

Caso de Éxito: Siemens — La Fábrica de Amberg

🏭 La fábrica de Siemens en Amberg (Alemania) es considerada una de las más avanzadas del mundo. Produce controladores industriales SIMATIC con un nivel de automatización del 75% y una tasa de defectos de solo 11.5 ppm (partes por millón). Cada producto es rastreado individualmente a lo largo de más de 1.000 estaciones de trabajo mediante IIoT.

Resultados: producción multiplicada por 13 en el mismo espacio desde 1989, con la misma cantidad de personal. Calidad del 99.99885%. 50 millones de datos de proceso analizados diariamente.

Tecnologías IIoT Utilizadas

- Gemelos digitales de cada producto y línea de producción.
- MES (Siemens SIMATIC IT) integrado con PLCs en tiempo real.
- OPC-UA para comunicación entre equipos de diferentes generaciones.
- Machine vision con IA para inspección de calidad automática.
- Dashboard central con OEE en tiempo real por línea.



Caso de Éxito: Caterpillar — Mantenimiento Predictivo en Minería

 Caterpillar, líder mundial en maquinaria pesada, implementó su plataforma Cat® Connect para monitorear más de 500.000 activos conectados globalmente. En minería, donde una hora de parada de un camión de 400 toneladas puede costar más de USD 10.000, el mantenimiento predictivo es fundamental.

500K+

activos conectados
globalmente

\$10K/h

costo de parada
de camión minero

30%

reducción de paradas
no planificadas

Tecnologías y resultados

Sensores de vibración, temperatura y presión de aceite en motores y transmisiones. Datos transmitidos vía satélite desde minas remotas sin cobertura celular. Modelos de ML entrenados con décadas de datos históricos de fallas. El sistema predice la vida útil restante (RUL) de componentes críticos como turbocompresores, inyectores y rodamientos de rueda. Resultado: los operadores mineros planifican el mantenimiento en ventanas de baja producción, eliminando paradas no planificadas que antes costaban millones anuales.



Caso de Éxito: Toyota – Lean Manufacturing + IIoT

 Toyota, creadora del sistema de producción que revolucionó la manufactura mundial (TPS – Toyota Production System), integró IIoT para llevar su filosofía Lean al siguiente nivel. Su planta de Motomachi en Japón combina los principios de Jidoka (automatización inteligente) y Just-in-Time con sensores IoT y analítica avanzada.

IIoT + Lean: la combinación potente

Andon digital: las alertas de piso de planta se digitalizaron. Cuando un operador detecta un defecto, el sistema IoT registra la estación, el producto, el tipo de error y la respuesta en tiempo real.

Poka-yoke conectado: los dispositivos anti-error ahora reportan cada evento al MES, permitiendo análisis de tendencia de errores humanos por estación y turno.

Kaizen basado en datos: las reuniones diarias de mejora continua ahora se basan en dashboards con OEE, tiempos de ciclo y tasas de defecto en tiempo real, no en reportes del día anterior.

Resultados medibles

- Tiempo de respuesta a anomalías reducido de minutos a segundos gracias a alertas IoT en tiempo real.
- Trazabilidad completa de cada vehículo: sensores registran más de 3.000 puntos de datos por auto durante el ensamblaje.
- Reducción de desperdicios energéticos del 15% usando sensores de consumo por estación de trabajo.
- Integración de robots colaborativos (cobots) con sensores de fuerza y proximidad IoT para trabajo seguro junto a operadores humanos.

7. LABORATORIO PBL

LAB 1 – Monitoreo IoT y Cálculo de KPIs de Motor

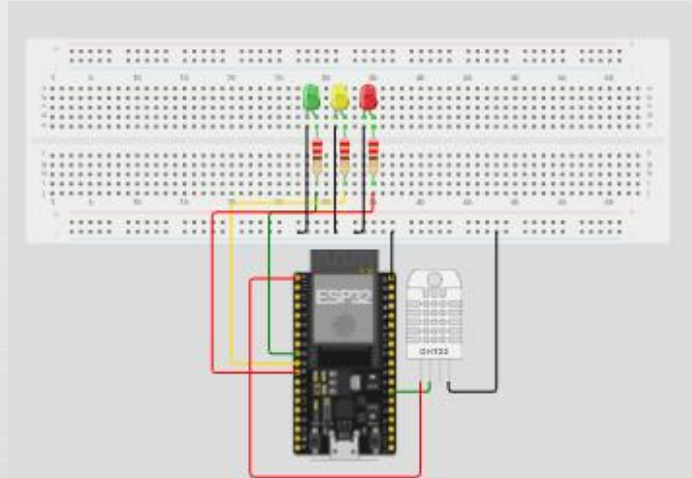
🔥 EN EQUIPOS — 45 MIN EN CLASE

📄 **ESCENARIO:** El jefe de mantenimiento necesita un sistema de monitoreo con cálculo automático de OEE y MTBF

La planta tiene un compresor crítico que ha tenido 6 paradas no planificadas en los últimos 3 meses. El jefe de mantenimiento quiere: (1) monitoreo de temperatura en tiempo real con alertas de 3 niveles, (2) registro de eventos de falla para calcular MTBF y MTTR, y (3) un dashboard que muestre el OEE del equipo. Su equipo debe diseñar la solución completa.

🔄 Entregables del LAB

1. Código en Wokwi: ESP32 + DHT22 + 3 LEDs monitoreando motor con alertas MQTT.
2. Cálculo de KPIs en Python (Colab): OEE, MTBF, MTTR y Disponibilidad con datos simulados.
3. Diagrama de arquitectura de la solución (sensor → plataforma → dashboard).



LAB 1 – Guía de Desarrollo Paso a Paso

Implementar en Wokwi

15 min

Usar código del ejercicio como base.
Adaptar: simular compresor industrial.
Verificar 3 niveles de alarma con LEDs.
Publicar telemetría MQTT.

Generar datos históricos

5 min

En Python (Colab): generar 30 días de datos simulados de temperatura con fallas inyectadas (6 eventos de parada).

Calcular MTBF y MTTR

10 min

Identificar las 6 fallas en los datos. Calcular tiempo entre fallas (MTBF) y tiempo de reparación de cada una (MTTR).

Calcular OEE

5 min

Disponibilidad × Rendimiento × Calidad.
Usar datos simulados del turno. Comparar con el benchmark de 85%.

Diagrama de arquitectura

5 min

Dibujar: sensor → ESP32 → MQTT → plataforma (ThingsBoard) → dashboard.

LAB 1 • Código Arduino IDE — Compresor con KPIs



Código Completo — Adaptar del ejercicio para LAB PBL

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT22
#define LED_G 25
#define LED_Y 26
#define LED_R 27
const char* ssid = "Wokwi-GUEST";
const char* pass = "";
const char* broker = "broker.hivemq.com";
const char* topic = "iot-fpuna/sem4/kpi/plant1a";
const char* equipo_id = "COMPRESOR-CRIT-01";
WiFiClient espClient;
PubSubClient mqtt(espClient);
DHT dht(DHTPIN, DHTTYPE);
int seq=0;
unsigned long t_operativo=0; // segundos operando
unsigned long t_parada=0; // segundos en parada
int fallas=0;
bool en_falla=false;
void setup() {
  Serial.begin(115200); dht.begin();
  pinMode(LED_G, OUTPUT);
  pinMode(LED_Y, OUTPUT);
  pinMode(LED_R, OUTPUT);
  WiFi.begin(ssid, pass);
  while(WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("WiFi OK");
  mqtt.setServer(broker, 1883);
}
void reconectar(){
  while(!mqtt.connected()){
    if(mqtt.connect(equipo_id))
      Serial.println("MQTT OK");
    else delay(3000);
  }
}
void loop() {
  if(!mqtt.connected()) reconectar();
  mqtt.loop();
  float t=dht.readTemperature();
  float h=dht.readHumidity();
```

```
if(isnan(t)){delay(2000);return;}
seq++;
const char* estado;
if(t>=75.0){
  estado="ALARMA";
  digitalWrite(LED_G,LOW);
  digitalWrite(LED_Y,LOW);
  digitalWrite(LED_R,HIGH);
  if(!en_falla){fallas++;en_falla=true;}
  t_parada += 3;
}else if(t>=60.0){
  estado="ADVERTENCIA";
  digitalWrite(LED_G,LOW);
  digitalWrite(LED_Y,HIGH);
  digitalWrite(LED_R,LOW);
  en_falla=false;
  t_operativo += 3;
}else{
  estado="NORMAL";
  digitalWrite(LED_G,HIGH);
  digitalWrite(LED_Y,LOW);
  digitalWrite(LED_R,LOW);
  en_falla=false;
  t_operativo += 3;
}
// Calcular disponibilidad en vivo
float disp = (t_operativo+t_parada>0) ?
  (float)t_operativo/(t_operativo+t_parada)*100 : 100;
float mtbf_h = (fallas>0) ?
  (float)t_operativo/3600.0/fallas : 0;
char json[350];
snprintf(json,350,
  "{\n\"equipo\": \"%s\", \"temp\": %.1f,\n"
  "\"estado\": \"%s\", \"fallas\": %d,\n"
  "\"disp_pct\": %.1f, \"mtbf_h\": %.1f,\n"
  "\"seq\": %d",
  equipo_id,t,estado,fallas,disp,mtbf_h,seq);
mqtt.publish(topic, json);
Serial.println(json);
delay(3000);
}
```

Notas Clave

Novedades vs. ejercicio:

$t_{operativo}$ / t_{parada}

Contadores de tiempo acumulado. Cada loop() suma 3 seg al contador correspondiente según el estado.

fallas (contador)

Se incrementa solo cuando la transición es de OK → ALARMA (flag en_falla evita contar múltiples veces la misma falla).

Disponibilidad en vivo:

$$disp = \frac{t_{operativo}}{(t_{operativo} + t_{parada})} \times 100$$

Se calcula y publica en cada lectura.

MTBF en vivo:

$$mtbf_h = \frac{(t_{operativo}/3600)}{fallas}$$

Horas promedio entre fallas. Se actualiza con cada nueva falla detectada.

LAB 1 • Código MicroPython — Compresor con KPIs



Código Completo — Copiar en main.py del ESP32 en Wokwi

```
import network, dht, machine, time, json
from umqtt.simple import MQTTClient

# == CONFIGURACIÓN ==
WIFI_SSID = 'Wokwi-GUEST'
BROKER = 'broker.hivemq.com'
TOPIC = 'iot-fpuna/sem4/kpi/planta1'
EQUIPO_ID = 'COMPRESOR-CRIT-01'
TEMP_WARN, TEMP_ALARM = 60.0, 75.0

# == HARDWARE ==
sensor = dht.DHT22(machine.Pin(4))
led_g = machine.Pin(25, machine.Pin.OUT)
led_y = machine.Pin(26, machine.Pin.OUT)
led_r = machine.Pin(27, machine.Pin.OUT)

# == WiFi ==
sta = network.WLAN(network.STA_IF)
sta.active(True)
sta.connect(WIFI_SSID, '')
while not sta.isconnected(): time.sleep(0.5)
print('WiFi OK:', sta.ifconfig()[0])

# == MQTT ==
client = MQTTClient(EQUIPO_ID, BROKER)
client.connect()
print('MQTT OK')

# == VARIABLES DE KPIs ==
seq = 0
t_operativo = 0 # segundos operando
t_parada = 0 # segundos en falla
fallas = 0
en_falla = False

while True:
    sensor.measure()

    t = sensor.temperature()
    h = sensor.humidity()
    seq += 1

    # Lógica de 3 niveles
    if t >= TEMP_ALARM:
        estado = 'ALARMA'
        led_g.off(); led_y.off(); led_r.on()
        if not en_falla:
            fallas += 1
            en_falla = True
            t_parada += 3
        elif t >= TEMP_WARN:
            estado = 'ADVERTENCIA'
            led_g.off(); led_y.on(); led_r.off()
            en_falla = False
            t_operativo += 3
        else:
            estado = 'NORMAL'
            led_g.on(); led_y.off(); led_r.off()
            en_falla = False
            t_operativo += 3

    # KPIs en vivo
    total = t_operativo + t_parada
    disp = t_operativo / total * 100 if total > 0 else 100
    mtbf = t_operativo / 3600 / fallas if fallas > 0 else 0

    payload = json.dumps({
        'equipo': EQUIPO_ID, 'temp': round(t,1),
        'estado': estado, 'fallas': fallas,
        'disp_pct': round(disp,1),
        'mtbf_h': round(mtbf,2), 'seq': seq
    })
    client.publish(TOPIC, payload)
    print(payload)
    time.sleep(3)
```

Diferencias vs Arduino

Nota importante:

MicroPython en Wokwi NO conecta a brokers MQTT externos. Usar este código como referencia y para hardware real. Para Wokwi, usar la versión Arduino IDE.

Equivalencias clave:

Arduino → MicroPython

digitalWrite() → led.on()/off()
dht.readTemp() → sensor.measure()
+ sensor.temperature()
sprintf(json) → json.dumps(dict)
mqtt.publish() → client.publish()
delay(3000) → time.sleep(3)

bool en_falla (Python):

Mismo patrón que Arduino. Evita contar la misma falla más de una vez mientras persiste.

LAB 1 • Código Python — Cálculo de KPIs en Google Colab



Código Completo — Copiar en Google Colab

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Generar datos simulados del compresor (30 días)
np.random.seed(42)
n = 8640 # lecturas cada 5 min x 30 días
tiempo = pd.date_range("2026-04-01", periods=n, freq="5min")
temp = np.random.normal(52, 5, n) # temp normal ~52°C

# Inyectar 6 fallas (temp sube a 80-95°C)
fallas_idx = [500, 1800, 3200, 5000, 6500, 8000]
for idx in fallas_idx:
    duracion = np.random.randint(5, 15)
    temp[idx:idx+duracion] = np.random.uniform(80, 95, duracion)

df = pd.DataFrame({
    "timestamp": tiempo,
    "equipo": "COMPRESOR-CRIT-01",
    "temp_c": np.round(temp, 1)
})
df["en_falla"] = df["temp_c"] >= 75.0

# 2. Identificar eventos de falla (inicio y fin)
df["cambio"] = df["en_falla"] != df["en_falla"].shift()
inicios = df[df["cambio"] & df["en_falla"]]
fines = df[df["cambio"] & ~df["en_falla"]]
print(f"Fallas detectadas: {len(inicios)}")

# 3. Calcular MTBF
t_total_h = len(df) * 5 / 60 # horas totales
t_falla_h = df["en_falla"].sum() * 5 / 60
t_operativo_h = t_total_h - t_falla_h
n_fallas = len(inicios)
MTBF = t_operativo_h / n_fallas
MTTR = t_falla_h / n_fallas

DISP = MTBF / (MTBF + MTTR) * 100
print(f"MTBF: {MTBF:.1f} h | MTTR: {MTTR:.2f} h")
print(f"Disponibilidad: {DISP:.2f}%")

# 4. Calcular OEE (simplificado)
disponibilidad = DISP / 100
rendimiento = 0.93 # dato del proceso
calidad = 0.985 # dato de calidad
OEE = disponibilidad * rendimiento * calidad * 100
print(f"OEE: {OEE:.1f}%")

# 5. Graficar serie temporal con fallas
plt.figure(figsize=(14, 4))
plt.plot(df["timestamp"], df["temp_c"],
         color="#00ACC1", linewidth=0.8, label="Temperatura")
plt.axhline(y=75, color="red", linestyle="--",
            label="Umbral alarma (75°C)")
plt.axhline(y=60, color="orange", linestyle="--",
            label="Umbral advertencia (60°C)")
plt.scatter(inicios["timestamp"], inicios["temp_c"],
            color="red", s=80, zorder=5, label="Inicio falla")
plt.title(f"COMPRESOR-CRIT-01 | OEE={OEE:.1f}%\n"
          f"| MTBF={MTBF:.0f}h | Disp={DISP:.1f}%")
plt.xlabel("Tiempo"); plt.ylabel("Temp (°C)")
plt.legend(loc="upper right"); plt.tight_layout()
plt.show()
```

Explicación Clave

Simulación de fallas:

`fallas_idx = [500, 1800, ...]`

6 posiciones donde la temp sube a 80-95°C simulando sobrecalentamiento real del compresor.

Detección automática:

`df['cambio'] = en_falla != en_falla.shift()`

Detecta transiciones: el momento exacto donde pasa de OK a falla. `shift()` compara con la fila anterior.

Fórmulas implementadas:

`MTBF = t_operativo / n_fallas`

`MTTR = t_falla / n_fallas`

`Disp = MTBF / (MTBF+MTTR)`

`OEE = Disp × Rend × Cal`

Gráfico final:

Muestra la serie temporal completa con los umbrales y marca el inicio de cada falla con un punto rojo. El título incluye los KPIs calculados.

LAB 2 — Tarea para EDUCA: Dashboard de KPIs con Python

TAREA INDIVIDUAL

ESCENARIO: El gerente de planta quiere un reporte semanal de KPIs para 3 equipos críticos

El dataset CSV (en EDUCA) contiene registros de operación de 3 equipos durante 30 días: tiempos operativos, paradas, producción real vs. teórica, y piezas defectuosas. Calcular OEE, MTBF, MTTR y disponibilidad para cada equipo, generar gráficos comparativos y redactar recomendaciones al gerente.

Requisitos Técnicos

- 1. Cargar CSV en Google Colab con pandas.
- 2. Calcular OEE por equipo ($\text{Disp} \times \text{Rend} \times \text{Cal}$).
- 3. Calcular MTBF y MTTR por equipo.
- 4. Gráfico de barras comparativo de OEE (3 equipos).
- 5. Gráfico de tendencia de disponibilidad semanal.
- 6. Identificar equipo con peor rendimiento y explicar por qué.
- 7. Párrafo de recomendaciones (200 palabras) al gerente.

Entregables (subir a EDUCA)

- A) Notebook (.ipynb) con código ejecutado y gráficos.
- B) Tabla resumen de KPIs por equipo (OEE, MTBF, MTTR, Disponibilidad).
- C) Gráfico comparativo de OEE y gráfico de tendencia.
- D) Párrafo de recomendaciones al gerente indicando qué equipo priorizar y por qué.

LAB 2 — Guía Paso a Paso para la Tarea EDUCA

⚡ TAREA INDIVIDUAL — Seguir estos pasos

Descargar y cargar CSV

5 min

Descargar equipos_kpi_30dias.csv desde EDUCA. Subir a Google Colab (icono carpeta → Subir).
import pandas as pd
df = pd.read_csv('equipos_kpi_30dias.csv')
df.head()

Explorar el dataset

5 min

```
df.shape # filas y columnas
df['equipo_id'].unique() # 3 equipos
df.columns # ver todas las columnas
df.dtypes # tipos de datos
df.info() # resumen completo
```

Calcular OEE por equipo

10 min

```
for eq in df['equipo_id'].unique():
    sub = df[df['equipo_id']==eq]
    disp = sub['t_operativo'].sum() /
    sub['t_planificado'].sum()
    rend = sub['prod_real'].sum() /
    sub['prod_teorica'].sum()
    cal = sub['piezas_ok'].sum() /
    sub['prod_real'].sum()
    oee = disp * rend * cal * 100
    print(f'{eq}: OEE={oee:.1f}%')
```

Calcular MTBF y MTTR

10 min

```
for eq in df['equipo_id'].unique():
    sub = df[df['equipo_id']==eq]
    t_op = sub['t_operativo'].sum()
    t_rep = sub['t_reparacion'].sum()
    n_fallas = sub['n_fallas'].sum()
    mtbf = t_op / n_fallas
    mttr = t_rep / n_fallas
    disp = mtbf/(mtbf+mttr)*100
    print(f'{eq}: MTBF={mtbf:.0f}h')
```

Gráficos comparativos

10 min

```
# Barras de OEE
plt.bar(equipos, oee_valores,
        color=['#00ACC1','#E8912D','#2E7D32'])
plt.axhline(y=85, linestyle='--',
            color='red', label='Clase mundial')
plt.title('OEE por Equipo')
plt.ylabel('OEE (%)')
```

Conclusiones (200 palabras)

5 min

Responder al gerente:

1. ¿Cuál equipo tiene peor OEE?
2. ¿La causa es disponibilidad, rendimiento o calidad?
3. ¿Qué equipo priorizar para mantenimiento predictivo?
4. Recomendación concreta.

LAB 2 • Código Python — Plantilla para la Tarea EDUCA



Código Base — Copiar y completar en Google Colab

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. CARGAR DATOS
df = pd.read_csv('equipos_kpi_30dias.csv')
print(f"Dataset: {df.shape[0]} filas, {df.shape[1]} columnas")
print(f"Equipos: {df['equipo_id'].unique()}")

# 2. CALCULAR KPIS POR EQUIPO
resultados = []
for eq in df['equipo_id'].unique():
    sub = df[df['equipo_id'] == eq]
    # OEE
    disp = sub['t_operativo'].sum() / sub['t_planificado'].sum()
    rend = sub['prod_real'].sum() / sub['prod_teorica'].sum()
    cal = sub['piezas_ok'].sum() / sub['prod_real'].sum()
    oee = disp * rend * cal * 100
    # MTBF / MTTR
    n_fallas = sub['n_fallas'].sum()
    mtbf = sub['t_operativo'].sum() / n_fallas if n_fallas > 0
    else 0
    mttr = sub['t_reparacion'].sum() / n_fallas if n_fallas > 0
    else 0
    disp_pct = mtbf / (mtbf + mttr) * 100 if (mtbf+mttr) > 0 else 0
    resultados.append({
        'Equipo': eq, 'OEE (%)': round(oee,1),
        'MTBF (h)': round(mtbf,1), 'MTTR (h)': round(mttr,2),
        'Disponibilidad (%)': round(disp_pct,1)
    })
tabla = pd.DataFrame(resultados)
print(tabla.to_string(index=False))

# 3. GRÁFICO DE BARRAS — OEE COMPARATIVO
fig, ax = plt.subplots(figsize=(8, 4))
colores = ['#00ACC1', '#E8912D', '#2E7D32']

ax.bar(tabla['Equipo'], tabla['OEE (%)'], color=colores)
ax.axhline(y=85, color='red', linestyle='--', label='Clase mundial 85%')
ax.set_ylabel('OEE (%)'); ax.set_title('OEE Comparativo por Equipo')
ax.legend(); plt.tight_layout(); plt.show()

# 4. GRÁFICO DE TENDENCIA SEMANAL
df['semana'] = pd.to_datetime(df['fecha']).dt.isocalendar().week
for eq in df['equipo_id'].unique():
    sub = df[df['equipo_id'] == eq]
    semanal = sub.groupby('semana').apply(
        lambda x: x['t_operativo'].sum() /
        x['t_planificado'].sum() * 100)
    plt.plot(semanal.index, semanal.values, marker='o', label=eq)
plt.xlabel('Semana'); plt.ylabel('Disponibilidad (%)')
plt.title('Tendencia de Disponibilidad Semanal')
plt.legend(); plt.tight_layout(); plt.show()
```

Explicación Clave

Estructura del CSV:

Columnas del archivo:

fecha, equipo_id,
t_planificado, t_operativo,
t_reparacion, n_fallas,
prod_teorica, prod_real,
piezas_ok

Cada fila = 1 día de operación de 1 equipo (90 filas total: 3 equipos × 30 días).

Fórmulas usadas:

$$OEE = (t_{op}/t_{plan}) \times (prod_{real}/prod_{teorica}) \times (piezas_{ok}/prod_{real})$$

Tendencia semanal:

groupby('semana').apply() agrupa los datos por semana y calcula disponibilidad. El gráfico de línea muestra si el equipo mejora o empeora con el tiempo.

8. RESUMEN, BIBLIOGRAFÍA Y PRÓXIMOS PASOS

Resumen — Ideas Clave de la Semana 4

4 IDEAS CLAVE QUE DEBES LLEVAR DE HOY

1

IloT no es IoT con otro nombre — es IoT en entorno crítico

Tolerancia cero a fallos, ciclos de vida de 10-30 años, protocolos industriales (OPC-UA, Modbus), convergencia IT/OT. El IloT opera donde una falla puede detener una planta entera o poner vidas en riesgo.

2

Los KPIs son el lenguaje que conecta el piso de planta con la gerencia

OEE = Disponibilidad × Rendimiento × Calidad. MTBF y MTTR miden confiabilidad y mantenibilidad. Sin KPIs no hay forma de justificar inversiones ni medir mejoras. IoT permite calcularlos en tiempo real.

3

El mantenimiento predictivo es el caso de negocio más fuerte del IloT

De correctivo (reparar cuando falla) a predictivo (reparar cuando los datos lo indican). Reducción de paradas 30-50%, extensión de vida útil 20-40%. Requiere pipeline de datos + ML, todo lo que vimos en la Semana 3.

4

Los casos reales demuestran ROI medible y escalable

Siemens Amberg: 99.99885% calidad con IloT. Caterpillar: 500K+ activos conectados. Toyota: Lean + IoT = Kaizen basado en datos. La inversión se justifica con los KPIs.

 **PRÓXIMOS PASOS:** Semana 5: Ciudades, Energía y Edificios Inteligentes — Smart grid, PUE y sensores urbanos

Referencias



Bibliografía — Semana 4: Smart Manufacturing e IIoT

Textos Base

Kranz, M. (2016). Building the Internet of Things. Wiley. Capítulos sobre manufactura y modelos de servicio.

Kagermann, H., Wahlster, W. & Helbig, J. (2013). Recommendations for Implementing the Strategic Initiative Industrie 4.0. Acatech.

Lasi, H. et al. (2014). 'Industry 4.0'. Business & Information Systems Engineering, 6(4), pp. 239-242.

Bahga, A. & Madiseti, V. (2014). Internet of Things: A Hands-On Approach. VPT.

Casos de Éxito y Estándares

Siemens AG. Amberg Electronics Plant. [siemens.com/amberg](https://www.siemens.com/amberg).

Caterpillar Inc. Cat® Connect Technology. [cat.com/connect](https://www.cat.com/connect).

Toyota Motor Corp. Toyota Production System Overview.

OPC Foundation. OPC-UA Specification. [opcfoundation.org](https://www.opcfoundation.org).

ISA-18.2. Management of Alarm Systems. [isa.org](https://www.isa.org).

Nakajima, S. (1988). Introduction to TPM. Productivity Press.



¡MUCHAS GRACIAS!